Robotics 2

Lecture Notes

Flavio Massaroni

Last updated: March 4, 2026

Contents

Preface & Acknowledgments **2**

Lecture 1 (3/03) – *Kinematic Calibration* **2**

1.1 Introduction 2
1.2 Calibration Procedure 3
1.3 Calibration Algorithm 4
1.4 Under- and Over-determined Systems 5
1.5 Final Comments 7

Lecture 2 (4/03) – *Lecture2* **7**

2.1 Introduction 7
2.2 Introduction 7

Preface & Acknowledgments

These lecture notes are based entirely on the concepts, lectures, and slide materials provided by **Prof. Alessandro De Luca** for the *Robotics 2* course at Sapienza University of Rome (Academic Year 2025/2026).

Unless otherwise specified, all equations, algorithms, and figures are directly adapted from the official course slides. The primary reference textbook for this course is:

- B. Siciliano, L. Villani, G. Oriolo, A. De Luca, *Foundations of Robotics*, Springer, September 2025.

All original course materials are the ones disposed by the professor on the official course website:
https://www.diag.uniroma1.it/~deluca/rob2_en.php.

※ Lecture 1

1.1 Introduction

For a robot manipulator in a given configuration, the actual values of the task-space variables may slightly deviate from those computed via direct kinematics. Thus it may lead to a deviation between the end-effector pose reached in a given configuration and the pose compute via direct kinematics. Such deviations characterize the *accuracy* of a robot manipulator (its ability to reproduce precisely the commanded pose in task space). The accuracy is different from *repeatability*, which is the robot capability to reproduce exactly the same commanded pose in more attempts. It depends more on the mechanical components' and control loops' quality.

The positional accuracy error may be reduced drastically by using the **kinematic calibration procedure**. The reasons why calibration is crucial in real-world conditions can be summarized by three principal problems:

- Mechanical tolerances;
- Encoder offsets (bias which make inconsistent the "zero reference" of the robot);

- Open kinematic chain loop (which amplifies the errors distributed along the arm).

Mathematically, let $\mathbf{y}_{e,nom} = \mathbf{k}_e(\boldsymbol{\alpha}, \mathbf{a}, \mathbf{d}, \boldsymbol{\theta})$ be the nominal Direct Kinematics (DK) mapping the joint space $\mathbf{q}$ to the task space $\mathbf{y}$, with constant geometric parameters. The calibration task consists of finding the true parameter values to decrease the discrepancy between the measured pose $\mathbf{y}_e$ and the nominal pose $\mathbf{y}_{e,nom}$:

$$\Delta y_e = y_e - y_{e,nom} = \frac{\partial k_e}{\partial \alpha} \cdot \Delta\alpha + \frac{\partial k_e}{\partial a} \cdot \Delta a + \frac{\partial k_e}{\partial d} \cdot \Delta d + \frac{\partial k_e}{\partial \theta} \cdot \Delta\theta$$

In the equation, the partial derivatives represent the partial Jacobians evaluated in nominal conditions and are multiplied by the first-order variations of the DH table values.

1.2 Calibration Procedure

To rewrite the linearization in a more compact matrix form, let's define the parameter variation vector $\Delta\varphi$ (of dimension $4n \times 1$) and the Jacobian matrix Φ (of dimension $6 \times 4n$, assuming full pose data):

$$\Delta\varphi = \begin{pmatrix} \Delta\alpha \\ \Delta a \\ \Delta d \\ \Delta\theta \end{pmatrix}, \quad \Phi = \begin{pmatrix} \frac{\partial k_e}{\partial \alpha} & \frac{\partial k_e}{\partial a} & \frac{\partial k_e}{\partial d} & \frac{\partial k_e}{\partial \theta} \end{pmatrix}$$

This allows us to express the single-pose calibration equation as:

$$\Delta y_e = \Phi \Delta\varphi$$

Since a single measurement is not enough to identify all the parameters, we need to perform ℓ experiments (with $\ell \gg n$). By stacking the measured pose errors and the corresponding matrices evaluated at different nominal configurations, we obtain:

$$\overline{\Delta y_e} = \begin{pmatrix} \Delta y_{e,1} \\ \Delta y_{e,2} \\ \vdots \\ \Delta y_{e,\ell} \end{pmatrix}_{6\ell \times 1}, \quad \overline{\Phi} = \begin{pmatrix} \Phi_1 \\ \Phi_2 \\ \vdots \\ \Phi_\ell \end{pmatrix}_{6\ell \times 4n}$$

The overall calibration equation for multiple experiments becomes a linear system:

$$\overline{\Delta y_e} = \overline{\Phi}\Delta\varphi$$

In this final system:

- $\overline{\Delta y_e}$ is the vector obtained from measurements;
- $\overline{\Phi}$ is the **regressor matrix** evaluated at nominal parameters. For the system to be solvable, this matrix must have full column rank (achievable for a sufficiently large ℓ);
- $\Delta\varphi$ is the vector of unknowns we want to estimate.

1.3 Calibration Algorithm

The overall calibration equation $\overline{\Delta y_e} = \overline{\Phi}\Delta\varphi$ represents a linearized least squares problem. Since the true direct kinematics function is non-linear, this linearized formulation needs an iterative solution, dictated by the following algorithm:

$\epsilon_r = 10^{-4}, \quad i_{max} = 50$

$\varphi^{(0)} = \varphi_{nom}$

$\overline{\Phi}^{(0)} = \overline{\Phi}(\varphi^{(0)})$

$\overline{\Delta y_e}^{(0)} = \overline{\Delta y_e}(\varphi^{(0)})$

$i = 0$

if $\left\|\overline{\Delta y_e}^{(i)}\right\| \leq \epsilon_r$

  $\varphi^* = \varphi^{(i)} \quad \leftarrow$ *final solution*

else if $i \leq i_{max}$

  $\Delta\varphi^{(i)} = \left(\overline{\Phi}^{(i)}\right)^{\#} \overline{\Delta y_e}^{(i)} = \left(\overline{\Phi}^{(i)T}\overline{\Phi}^{(i)}\right)^{-1} \overline{\Phi}^{(i)T}\overline{\Delta y_e}^{(i)}$

  $\varphi^{(i+1)} = \varphi^{(i)} + \Delta\varphi^{(i)}$

  $\overline{\Phi}^{(i+1)} = \overline{\Phi}(\varphi^{(i+1)})$

  $\overline{\Delta y_e}^{(i+1)} = \overline{\Delta y_e}(\varphi^{(i+1)})$

  $i = i + 1$

else

  `disp('no convergence in` i_{max} `iterations')`

end if

Algorithm Breakdown

To better understand the iterative procedure, we can break it down into the following key steps:

1. **Initialization:** We define a tolerance threshold (ϵ_r) and a maximum number of iterations (i_{max}) to prevent infinite loops. We start by guessing that our parameters are equal to the nominal ones ($\varphi^{(0)} = \varphi_{nom}$).

2. **Convergence Check:** At each step i, we evaluate the norm of the pose error $\left\|\overline{\Delta y_e}^{(i)}\right\|$. If this error is smaller than our tolerance ϵ_r, the algorithm stops, and the current parameters are accepted as the true calibrated parameters φ^*.

3. **Least Squares Minimization:** If the error is still too large, we compute the required parameter variation $\Delta\varphi^{(i)}$. Since the matrix $\overline{\Phi}$ is rectangular (overdetermined system), we use the **Moore-Penrose pseudoinverse** $\left(\overline{\Phi}^{(i)}\right)^{\#}$. This specific mathematical operation finds the $\Delta\varphi^{(i)}$ that minimizes the squared error norm $\left\|\overline{\Phi}^{(i)}\Delta\varphi - \overline{\Delta y_e}^{(i)}\right\|^2$.

4. **Update Rule:** The calculated variation $\Delta\varphi^{(i)}$ is added to the current parameters to generate a new, more accurate guess $\varphi^{(i+1)}$. This new set contains the updated Denavit-Hartenberg parameters and the newly estimated encoder offsets (bias on θ).

5. **Re-evaluation:** The regressor matrix $\overline{\Phi}$ and the pose error $\overline{\Delta y_e}$ are re-evaluated using the newly found parameters $\varphi^{(i+1)}$, and the loop repeats.

1.4 Under- and Over-determined Systems

Different solutions may be found when considering **Under-determined** and **Over-determined** systems of equations.

In the context of the calibration algorithm, we evaluate linear systems in the standard form $Ax = b$. For our calibration, the system is $\overline{\Phi}\Delta\varphi = \overline{\Delta y_e}$. Depending on the number of equations versus the number of unknowns, we face two

different scenarios. Both are solved using the **Moore-Penrose pseudoinverse** $A^{\#}$:

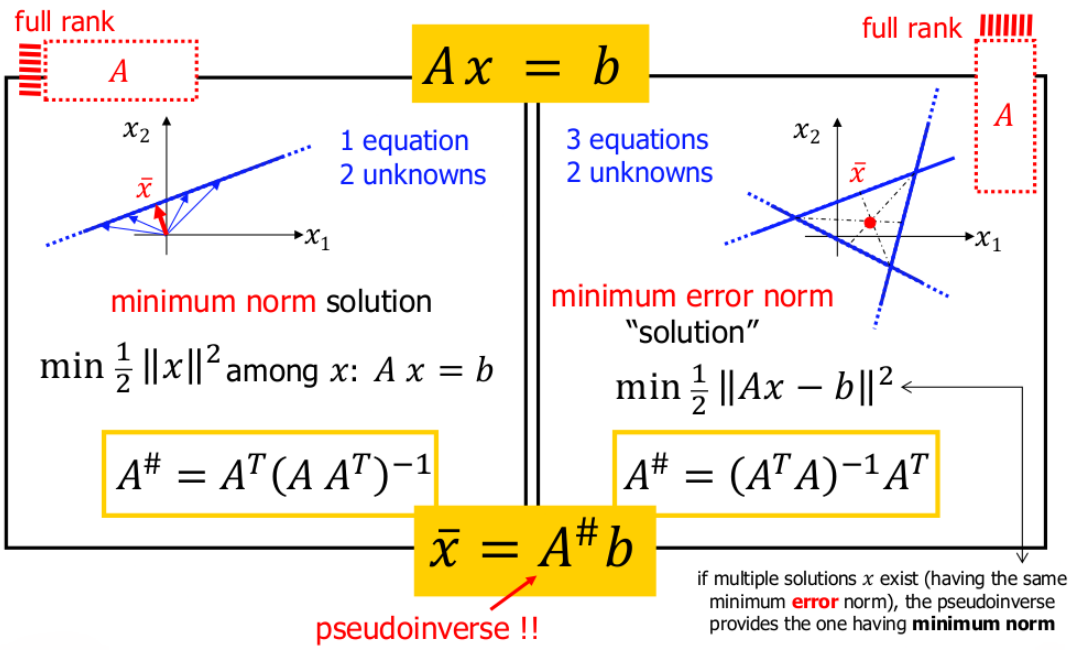

Figure 1: Geometric interpretation of under-determined and over-determined linear systems.

As illustrated in Figure 1, the pseudoinverse handles two main configurations:

- **Under-determined system (left):** There are fewer equations than unknowns (e.g., 1 equation, 2 unknowns). The matrix A has *full row rank*. Since there are infinite exact solutions, we look for the **minimum norm** solution, which minimizes $\frac{1}{2}\|x\|^2$ among all x such that $Ax = b$. The pseudoinverse used is:

$$A^{\#} = A^T(AA^T)^{-1}$$

- **Over-determined system (right):** There are more equations than unknowns (e.g., 3 equations, 2 unknowns). The matrix A has *full column rank*. Because measurements contain noise, an exact intersection usually does not exist. Instead, we look for the **minimum error norm** solution, minimizing the squared error $\frac{1}{2}\|Ax - b\|^2$. The pseudoinverse used is:

$$A^{\#} = (A^T A)^{-1} A^T$$

In both scenarios, the optimal solution is compactly written as:

$$\bar{x} = A^{\#} b$$

Note: If multiple solutions exist with the same minimum error norm, the pseudoinverse automatically provides the one having the **minimum norm**.

Connection to Calibration: Since we perform ℓ experiments where $\ell \gg n$, our calibration problem falls strictly into the **over-determined** category. This justifies why our iterative algorithm calculates the parameter variation using the left-pseudoinverse formulation: $\Delta\varphi^{(i)} = \left(\overline{\Phi}^{(i)T}\overline{\Phi}^{(i)}\right)^{-1}\overline{\Phi}^{(i)T}\overline{\Delta y_e}^{(i)}$.

1.5 Final Comments

The kinematic calibration procedure is an iterative least squared method, which starts as a non-linear problem and it is linearized by using the first-order Taylor expansion. It represent an important problem in modern Robotics, to accurate calibrate and estimate real parameters for sensors and model is foundamental to increase accuracy.

※ Lecture 2

2.1 Introduction

albero

2.2 Introduction

References

[1] B. Siciliano, L. Villani, G. Oriolo, A. De Luca. *Foundations of Robotics*. Springer, September 2025.

[2] A. De Luca. "Robotics 2 Course Material and Slides". Sapienza University of Rome, Academic Year 2025/2026. https://www.diag.uniroma1.it/~deluca/rob2_en.php.